

Assignment 02: การเรียนรู้ Arrays และ Loop Structures สำหรับ Game Development

จุดประสงค์การเรียนรู้

- เรียนรู้การประกาศ การใช้งาน และการจัดการ Arrays
- เข้าใจ for loop และ while loop structures
- นำ Arrays และ Loops มาใช้ร่วมกันในการแก้ปัญหา
- จัดการ array indexing และ array boundaries อย่างปลอดภัย
- นำ Arrays และ Loops มาใช้ในสถานการณ์ game development จริง
- เขียน code ที่สะอาด อ่านง่าย และปฏิบัติตาม best practices

โครงสร้างของ Assignment

- **Lecture Methods (7 methods)** - การ implement ฝึกหัดด้วย Arrays และ Loops พื้นฐาน พร้อมกันในห้องเรียน
- **Assignment Methods (16 methods)** - การประยุกต์ใช้ Arrays และ Loops ในสถานการณ์เกม
- **Extra Assignment Method (1 method)** - โจทย์ขั้นสูง (ไม่บังคับ)

ตัวแปรบน Inspector (สำคัญ)

ทุก method ใน `Lecture.cs` และ `Assignment.cs` ไม่รับ parameter อีกต่อไป — ค่าที่เคยเป็น parameter ของแต่ละ method ได้ถูกย้ายไปเป็น public field ของคลาสแทน เพื่อให้กำหนดค่าทดสอบได้จากหน้า Inspector ของ Unity โดยตรง

- แต่ละ field จะขึ้นต้นด้วย prefix ของ method นั้นๆ เช่น method `AS04_AttackEnemy` จะมี field `as04_enemyHP`, `as04_damage`, `as04_target`
- ใน Inspector จะมี [Header] คั่นเป็นกลุ่มๆ ตามชื่อ method เพื่อให้แยกกลุ่มตัวแปรของแต่ละโจทย์ได้ง่าย
- เอกสารด้านล่างจะเขียน Method Signature แบบไม่มี parameter `()` และแจกแจง field ที่เกี่ยวข้องแยกไว้ในหัวข้อ **ตัวแปร (Inspector Fields)** ของแต่ละ method แทน

2D Array บน Inspector: Grid2DInt / Grid2DString

Unity ไม่รองรับการแสดงผล `int[,]` หรือ `string[,]` (rectangular 2D array) บน Inspector โดยตรง ดังนั้น method ที่ได้รับ 2D array เป็น parameter (LCT06, AS13, AS14, EX_01) จะใช้ field ชนิด `Grid2DInt` หรือ `Grid2DString` แทน ซึ่งเป็น class ที่เขียน custom Inspector ให้กรอกค่าเป็นตาราง (grid) ได้ตรงๆ

เมื่อจะนำไปใช้งานเป็น 2D array จริงในโค้ด ให้เรียก `.Get2DArray()`:

```
int[,] arr = lct06_my2DArray.Get2DArray(); // สำหรับ Grid2DInt
string[,] board = ex01_board.Get2DArray(); // สำหรับ Grid2DString
```

Lecture Methods

Methods เหล่านี้แสดงแนวคิด Arrays และ Loops พื้นฐาน Implement เพื่อฝึกหัดแต่จะไม่มีการให้คะแนน

1. LCT01_SyntaxArray

วัตถุประสงค์: แสดงการประกาศและใช้งาน Array พื้นฐาน วิธี Get, Set และเข้าถึงขนาดของ Array

Method Signature:

```
void LCT01_SyntaxArray()
```

Logic ที่ต้อง implement:

- สร้าง string array ขนาด 2 ช่อง (หรือ 3 ช่อง) ชื่อ `ironManSuit`
- Set กำหนดค่า: `ironManSuit[0] = "Mark I", ironManSuit[1] = "Mark II"`
- สร้างตัวแปร `tonyStarkWear` ดึงค่าจาก `ironManSuit[0]` เพื่อพิมพ์ข้อความ `TonyStark Wear: {tonyStarkWear}`
- Get ขนาดของ array ด้วย `.Length` และแสดงข้อความ `Room size: {ironManSuit.Length}`
- ทำการ Log ค่าของ `ironManSuit` ในช่องที่ 0 และ 1 (`Mark I, Mark II`)
- แสดงผลค่าต่างๆ ตามรูปแบบที่กำหนด

Test Case:

- **Input:** ไม่มี parameters
- **Expected Output:**

```
TonyStark Wear: Mark I
Room size: 2
Mark I
Mark II
```

2. LCT02_ArrayInitialize

วัตถุประสงค์: แสดงการประกาศ array แบบกำหนดขนาดและกำหนดค่าเริ่มต้น (Array Initialization) และการเข้าถึงข้อมูลใน array

Method Signature:

```
void LCT02_ArrayInitialize()
```

Logic ที่ต้อง implement:

- สร้างชุดข้อมูล array ของ Spider-Man suits โดยกำหนดค่าเริ่มต้นดังต่อไปนี้: `"Classic SpiderMan", "Black Suit", "Iron Spider Suit"`
- สร้างชุดข้อมูล array ของ Batman suits โดยกำหนดให้มีขนาดเท่ากับ 2 และมีค่าเริ่มต้นดังต่อไปนี้: `"Classic BatMan", "White bat"`
- ใช้ `array.Length` เพื่อแสดงขนาดของ array (`Room size: {array.Length}`)
- พิมพ์ข้อมูลของ array ทั้ง 2 ชุดตามลำดับ

Test Case:

- **Input:** ไม่มี parameters
- **Expected Output:**

```
Room size: 3
Classic SpiderMan
Black Suit
Iron Spider Suit
Room size: 2
Classic BatMan
White bat
```

3. LCT03_SyntaxLoop

วัตถุประสงค์: แสดงการใช้งานโครงสร้างการวนซ้ำ **for** loop พื้นฐาน

Method Signature:

```
void LCT03_SyntaxLoop()
```

Logic ที่ต้อง implement:

- **for loop ที่ 1:**
 - วนลูปทั้งหมด 10 ครั้ง โดยค่าของ **i** เริ่มต้นที่ 0 และเพิ่มขึ้นทีละ 1 จนถึงค่าน้อยกว่า 10 (**i = 0** ถึง **i < 10**)
 - ในแต่ละรอบของลูป แสดงข้อความ "<10 : " + **i** ออกมาทาง **Debug.Log**
- ก่อนเริ่ม for loop ที่ 2 ให้พิมพ์ **Debug.Log("=====");**
- **for loop ที่ 2:**
 - วนลูปทั้งหมด 10 ครั้ง โดยค่าของ **i** เริ่มต้นที่ 1 และเพิ่มขึ้นทีละ 1 จนถึงค่าเท่ากับ 10 (**i = 1** ถึง **i <= 10**)
 - ในแต่ละรอบของลูป แสดงข้อความ "<=10 : " + **i** ออกมาทาง **Debug.Log**

Expected Output:

```
<10 : 0
<10 : 1
<10 : 2
<10 : 3
<10 : 4
<10 : 5
<10 : 6
<10 : 7
<10 : 8
<10 : 9
=====
<=10 : 1
<=10 : 2
<=10 : 3
```

```
<=10 : 4
<=10 : 5
<=10 : 6
<=10 : 7
<=10 : 8
<=10 : 9
<=10 : 10
```

4. LCT04_LoopAndArray

วัตถุประสงค์: แสดงการใช้งาน Array ร่วมกับ **for** loop

Method Signature:

```
void LCT04_LoopAndArray()
```

ตัวแปร (Inspector Fields):

- `lct04_ironManSuitNames (string[])` - อาร์เรย์ของชื่อชุดเกราะ Iron Man

Logic ที่ต้อง implement:

- พิมพ์ข้อความ `Debug.Log("==== Log by One incrementer ====");`
- **for loop ที่ 1:** ค่า `i` เพิ่มขึ้นทีละ 1 เพื่อแสดงชื่อชุดเกราะทั้งหมดใน `lct04_ironManSuitNames`
- พิมพ์ข้อความ `Debug.Log("==== Log by Two incrementer ====");`
- **for loop ที่ 2:** ค่า `i` เพิ่มขึ้นทีละ 2 เพื่อแสดงชื่อชุดเกราะทุกๆ 2 ตำแหน่ง

Test Case:

- **Input:** `lct04_ironManSuitNames = ["Mark I", "Mark II", "Mark III", "Mark IV", "Mark V", "Mark VI", "Mark VII"]`
- **Expected Output:**

```
==== Log by One incrementer ====
Mark I
Mark II
Mark III
Mark IV
Mark V
Mark VI
Mark VII
==== Log by Two incrementer ====
Mark I
Mark III
Mark V
Mark VII
```

5. LCT05_Syntax2DArray

วัตถุประสงค์: แสดงการประกาศและสร้างอาร์เรย์สองมิติ (2D Array) พร้อมกำหนดค่าเริ่มต้น

Method Signature:

```
void LCT05_Syntax2DArray()
```

Logic ที่ต้อง implement:

- สร้าง 2D array ขนาด 3 x 3 ชื่อ `my2DArray` โดยมีค่าเริ่มต้นดังนี้: { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } }
- แสดงผลลัพธ์ข้อมูลใน 2D array แต่ละแถวออกมาทาง `Debug.Log`

Test Case:

- Input:** ไม่มี parameters
- Expected Output:**

```
1 2 3
4 5 6
7 8 9
```

6. LCT06_SizeOf2DArray

วัตถุประสงค์: แสดงการหาขนาดมิติต่างๆ ของอาร์เรย์สองมิติ (2D Array) ด้วย `.GetLength(dimension)`

Method Signature:

```
void LCT06_SizeOf2DArray()
```

ตัวแปร (Inspector Fields):

- `lct06_my2DArray (Grid2DInt)` - อาร์เรย์ 2 มิติ กรอกค่าเป็นตารางได้จาก Inspector เรียก `lct06_my2DArray.Get2DArray()` เพื่อแปลงเป็น `int[,]`

Logic ที่ต้อง implement:

- แปลง `lct06_my2DArray` เป็น `int[,]` ด้วย `lct06_my2DArray.Get2DArray()`
- ใช้ `array.GetLength(0)` เพื่อหาขนาดของมิติที่ 1 หรือจำนวนแถว (`rows`)
- ใช้ `array.GetLength(1)` เพื่อหาขนาดของมิติที่ 2 หรือจำนวนหลัก/คอลัมน์ (`cols`)
- แสดงผล Log ออกมาดังนี้:
 - `Debug.Log($"rows = {rows}");`
 - `Debug.Log($"cols = {cols}");`

Test Case:

- Input:** `lct06_my2DArray` ตั้งค่าเริ่มต้นเป็น 3 แถว x 5 หลัก (`rows = 3, cols = 5`)

- **Expected Output:**

```
rows = 3  
cols = 5
```

7. LCT07_SyntaxNestedLoop

วัตถุประสงค์: แสดงการใช้งาน Nested Loop (ลูปซ้อนลูป) เพื่อสร้าง pattern

Method Signature:

```
void LCT07_SyntaxNestedLoop()
```

ตัวแปร (Inspector Fields):

- `lct07_columns (int)` - จำนวนคอลัมน์ (หลัก)
- `lct07_rows (int)` - จำนวนแถว

Logic ที่ต้อง implement:

- ใช้ Nested Loop (ลูปแถวซ้อนลูปคอลัมน์) เพื่อสร้าง pattern ดาว (*) ตามขนาดที่กำหนด
- แต่ละแถวจะมีดาวจำนวน `columns` ดวง และมีจำนวนแถวทั้งหมด `rows` แถว

Test Cases:

1. **Input:** `lct07_columns = 3, lct07_rows = 4` **Expected Output:**

```
***  
***  
***  
***
```

2. **Input:** `lct07_columns = 10, lct07_rows = 1` **Expected Output:**

```
*****
```

3. **Input:** `lct07_columns = 10, lct07_rows = 10` **Expected Output:**

```
*****  
*****  
*****  
*****  
*****  
*****
```

```

*****
*****
*****
*****

```

4. **Input:** `lct07_columns = 5, lct07_rows = 3` **Expected Output:**

```

*****
*****
*****

```

5. **Input:** `lct07_columns = 1, lct07_rows = 5` **Expected Output:**

```

*
*
*
*
*

```

Assignment Methods

Methods เหล่านี้เป็นการประยุกต์ใช้ Arrays และ Loops ในสถานการณ์เกม และจะมีการให้คะแนน

AS01_RandomItemDrop

วัตถุประสงค์: สุ่มการดรอปไอเท็มจากรายการที่กำหนด และสร้าง GameObject (Instantiate) พร้อมแสดงชื่อไอเท็ม

Method Signature:

```
void AS01_RandomItemDrop()
```

ตัวแปร (Inspector Fields):

- `as01_items` (GameObject[]) - รายการของ GameObject ไอเท็มทั้งหมดที่จะสุ่มดรอป

Logic ที่ต้อง implement:

- สุ่มเลือก GameObject หนึ่งชิ้นจาก array `as01_items` โดยใช้ `UnityEngine.Random.Range(0, as01_items.Length)`
- ใช้ `Instantiate(selectedItem)` เพื่อสร้างออบเจกต์ในเกม
- แสดงชื่อไอเท็มที่สุ่มได้ออกมาทาง Console ในรูปแบบ `Debug.Log($"Got item: {go.name}");`

Test Cases:

1. **Input:** `as01_items = ["Sword", "Shield", "Potion"]` (3 items) **Expected Output:** Got item: {หนึ่งในไอเท็มที่กำหนด}
2. **Input:** `as01_items = ["Helmet"]` (1 item) **Expected Output:** Got item: Helmet
3. **Input:** `as01_items = ["Bow", "Arrow", "Quiver", "Magic Ring", "Health Potion"]` (5 items) **Expected Output:** Got item: {หนึ่งในไอเท็มที่กำหนด}

หมายเหตุ: ผลลัพธ์จะสุ่มจากรายการที่กำหนดใน Inspector ดังนั้นอาจได้อิเท็มใดก็ได้ในรายการ

AS02_NestedLoopForCreate2DMap

วัตถุประสงค์: สร้างแผนที่ 2D แบบสุ่มพื้นผิวด้วย Nested Loop

Method Signature:

```
void AS02_NestedLoopForCreate2DMap()
```

ตัวแปร (Inspector Fields):

- `as02_floorTiles` (`GameObject[]`) - อาร์เรย์ของ `GameObject` พื้นแบบต่างๆ (เช่น "0" แทนพื้นธรรมดา, "1" แทนพื้นแบบที่ 1, "2" แทนพื้นแบบที่ 2)
- `as02_columns` (`int`) - จำนวนคอลัมน์ของแผนที่
- `as02_rows` (`int`) - จำนวนแถวของแผนที่

Logic ที่ต้อง implement:

- ใช้ Nested Loop (ลูปแถวซ้อนลูปคอลัมน์) เพื่อสร้างแผนที่ขนาด `as02_columns` x `as02_rows`
- ในแต่ละตำแหน่ง (`x`, `y`) ให้สุ่มเลือก `GameObject` พื้นจาก array `as02_floorTiles`
- ใช้ `Instantiate(tilePrefab, new Vector2(x, y), transform.rotation)` เพื่อสร้างแผ่นพื้น
- แสดงชื่อ `GameObject` ของแผ่นพื้นออกมาเพื่อดู pattern ของแผนที่ที่สุ่มได้

Test Cases:

1. **Input:** `as02_floorTiles=["0", "1", "2"]`, `as02_columns=3`, `as02_rows=3` **Expected Behavior:** สร้าง `GameObject` จำนวน 9 ตัว (3x3) บนตำแหน่ง `Vector2(x, y)` **ตัวอย่าง Output:**

```
211
110
000
```

2. **Input:** `as02_floorTiles=["0", "1", "2"]`, `as02_columns=10`, `as02_rows=10` **Expected Behavior:** สร้าง `GameObject` จำนวน 100 ตัว (10x10) บนตำแหน่ง `Vector2(x, y)`

หมายเหตุ: การสุ่มทำให้ pattern ของแผนที่เปลี่ยนแปลงได้ แต่จำนวน `GameObject` ที่สร้างจะตรงกับ `columns` x `rows` เสมอ

AS03_NestedLoopForMakingWallAround

วัตถุประสงค์: สร้างกำแพงล้อมรอบนอกขอบของพื้นที่เล่นโดยใช้ Nested Loop

Method Signature:

```
void AS03_NestedLoopForMakingWallAround()
```

ตัวแปร (Inspector Fields):

- `as03_wall (GameObject)` - GameObject/Prefab กำแพง (ชื่อ "*")
- `as03_columns (int)` - จำนวนคอลัมน์ของพื้นที่เล่น
- `as03_rows (int)` - จำนวนแถวของพื้นที่เล่น

Logic ที่ต้อง implement:

- ใช้ Nested Loop วนตำแหน่ง `x` (คอลัมน์ 0 ถึง `as03_columns - 1`) และ `y` (แถว 0 ถึง `as03_rows - 1`)
- ตรวจสอบเงื่อนไขว่าตำแหน่งปัจจุบันอยู่ที่ขอบรอบนอกหรือไม่: `if (x == 0 || x == as03_columns - 1 || y == 0 || y == as03_rows - 1)`
 - ขอบบนสุด/Row แรก: `y == 0`
 - ขอบล่างสุด/Row สุดท้าย: `y == as03_rows - 1`
 - ขอบซ้ายสุด/Column แรก: `x == 0`
 - ขอบขวาสุด/Column สุดท้าย: `x == as03_columns - 1`
- หากเป็นตำแหน่งขอบ ให้สร้างกำแพงด้วย `Instantiate(as03_wall, new Vector2(x, y), transform.rotation)`

Test Cases:

1. **Input:** `as03_wall = GameObject("*")`, `as03_columns = 5`, `as03_rows = 3` **Expected Behavior:** สร้างกำแพงรอบขอบของพื้นที่ขนาด 5x3 **Pattern ตัวอย่าง:**

```
*****
*   *
*****
```

2. **Input:** `as03_wall = GameObject("*")`, `as03_columns = 3`, `as03_rows = 5` **Pattern ตัวอย่าง:**

```
***
* *
* *
* *
***
```

AS04_AttackEnemy

วัตถุประสงค์: ระบบคำนวณและลดค่า HP ของศัตรูจากการโจมตี 3 รูปแบบ

Method Signature:

```
void AS04_AttackEnemy()
```

ตัวแปร (Inspector Fields):

- `as04_enemyHP (int[])` - array ที่เก็บค่า HP ของ enemy แต่ละตัว
- `as04_damage (int)` - จำนวน damage ที่จะโจมตี
- `as04_target (int)` - index ของ enemy เป้าหมายที่จะโจมตี (สำหรับรูปแบบที่ 3)

Logic ที่ต้อง implement: โจมตีเรียงตามลำดับ 3 รูปแบบดังนี้:

1. **รูปแบบที่ 1 (โจมตีตัวแรก):** `as04_enemyHP[0] -= as04_damage` แล้ว Log `"FirstEnemy hp : {as04_enemyHP[0]}"`
2. **รูปแบบที่ 2 (โจมตีตัวสุดท้าย):** `as04_enemyHP[as04_enemyHP.Length - 1] -= as04_damage` แล้ว Log `"LastEnemy hp : {as04_enemyHP[as04_enemyHP.Length - 1]}"`
3. **รูปแบบที่ 3 (โจมตีเป้าหมายที่ระบุ):** `as04_enemyHP[as04_target] -= as04_damage` แล้ว Log `"TargetEnemy {as04_target} hp : {as04_enemyHP[as04_target]}"`

(หมายเหตุ: หาก HP ลดลงต่ำกว่า 0 ให้ปรับเป็น 0)

Test Cases:

1. **Input:** `as04_enemyHP = [10, 15, 20, 25, 30]`, `as04_damage = 2`, `as04_target = 3` **Expected Output:**

```
FirstEnemy hp :8
LastEnemy hp :28
TargetEnemy 3 hp :23
```

2. **Input:** `as04_enemyHP = [5, 10, 15]`, `as04_damage = 10`, `as04_target = 1` **Expected Output:**

```
FirstEnemy hp :0
LastEnemy hp :5
TargetEnemy 1 hp :0
```

3. **Input:** `as04_enemyHP = [20]`, `as04_damage = 5`, `as04_target = 0` **Expected Output:**

```
FirstEnemy hp :15
LastEnemy hp :10
TargetEnemy 0 hp :5
```

AS05_DynamicIterationLoop

วัตถุประสงค์: สร้าง `for` loop แบบไดนามิกตามจำนวนรอบ `as05_n` ที่กำหนด

Method Signature:

```
void AS05_DynamicIterationLoop()
```

ตัวแปร (Inspector Fields):

- `as05_n (int)` - ค่าจำนวนเต็มระบุจำนวนรอบที่จะวนลูป (จาก 0 ถึง $n - 1$)

Logic ที่ต้อง implement:

- สร้าง `for` loop จาก `i = 0` จนถึง `i < as05_n`
- แสดงผลค่าตัวเลข `i` ในแต่ละรอบออกมาทาง Console

Test Cases:

1. Input: `as05_n = 0` Expected Output: (ไม่มี output)

2. Input: `as05_n = 1` Expected Output:

```
0
```

3. Input: `as05_n = 3` Expected Output:

```
0
1
2
```

4. Input: `as05_n = 5` Expected Output:

```
0
1
2
```

AS06_WhileLoopAndArray

วัตถุประสงค์: แสดงรายชื่อชุดเกราะ Iron Man โดยใช้ Array และ `while` loop

Method Signature:

```
void AS06_WhileLoopAndArray()
```

ตัวแปร (Inspector Fields):

- `as06_ironManSuitNames (string[])` - อาร์เรย์ของชื่อชุดเกราะ Iron Man

Logic ที่ต้อง implement:

- พิมพ์หัวข้อ `Debug.Log("====Log by One====");`
- **while Loop ที่ 1:** ตัวนับ `i` เริ่มต้นที่ 0 และเพิ่มทีละ 1 (`i += 1`) เพื่อแสดงชื่อชุดเกราะทุกตัวตามลำดับ
- พิมพ์หัวข้อ `Debug.Log("====Log by Two====");`
- **while Loop ที่ 2:** ตัวนับ `i` เริ่มต้นที่ 0 และเพิ่มทีละ 2 (`i += 2`) เพื่อแสดงชื่อชุดเกราะทุกๆ 2 ตัว (index 0, 2, 4, ...)

Test Case:

- **Input:** `as06_ironManSuitNames = ["Mark I", "Mark II", "Mark III", "Mark IV", "Mark V", "Mark VI", "Mark VII"]`
- **Expected Output:**

```
====Log by One====
Mark I
Mark II
Mark III
Mark IV
Mark V
Mark VI
Mark VII
====Log by Two====
Mark I
Mark III
Mark V
Mark VII
```

AS07_HealTargetAtIndex

วัตถุประสงค์: ระบบคำนวณและเพิ่มค่า HP ของ Hero จากการ Heal 3 รูปแบบ

Method Signature:

```
void AS07_HealTargetAtIndex()
```

ตัวแปร (Inspector Fields):

- `as07_heroHPs (int[])` - array ที่เก็บค่า HP ของ hero แต่ละตัว
- `as07_heal (int)` - จำนวน heal ที่จะฟื้นฟู
- `as07_targetIndex (int)` - index ของ hero เป้าหมายที่จะฟื้นฟู (สำหรับรูปแบบที่ 3)

Logic ที่ต้อง implement: Heal เรียงตามลำดับ 3 รูปแบบดังนี้:

1. **รูปแบบที่ 1 (Heal ตัวแรก):** `as07_heroHPs[0] += as07_heal` แล้ว Log `$"FirstHero hp : {as07_heroHPs[0]}"`
2. **รูปแบบที่ 2 (Heal ตัวสุดท้าย):** `as07_heroHPs[as07_heroHPs.Length - 1] += as07_heal` แล้ว Log `$"LastHero hp : {as07_heroHPs[as07_heroHPs.Length - 1]}"`
3. **รูปแบบที่ 3 (Heal ตัวเป้าหมายที่กำหนด):** `as07_heroHPs[as07_targetIndex] += as07_heal` แล้ว Log `$"TargetHero {as07_targetIndex} hp : {as07_heroHPs[as07_targetIndex]}"`

Test Cases:

1. **Input:** `as07_heroHPs = [10, 15, 20, 25, 30]`, `as07_heal = 5`, `as07_targetIndex = 3`
Expected Output:

```
FirstHero hp :15
LastHero hp :35
TargetHero 3 hp :30
```

2. **Input:** `as07_heroHPs = [1, 2, 3]`, `as07_heal = 10`, `as07_targetIndex = 1` **Expected Output:**

```
FirstHero hp :11
LastHero hp :13
TargetHero 1 hp :12
```

3. **Input:** `as07_heroHPs = [100]`, `as07_heal = 50`, `as07_targetIndex = 0` **Expected Output:**

```
FirstHero hp :150
LastHero hp :200
TargetHero 0 hp :250
```

AS08_RandomPickingDialogue

วัตถุประสงค์: สร้างระบบสุ่มเลือกบทสนทนาจากชุดข้อความใน Array

Method Signature:

```
void AS08_RandomPickingDialogue()
```

ตัวแปร (Inspector Fields):

- `as08_dialogues (string[])` - Array ที่เก็บชุดข้อความบทสนทนาทั้งหมด

Logic ที่ต้อง implement:

- สุ่มดัชนีโดยใช้ `int r = UnityEngine.Random.Range(0, as08_dialogues.Length);` (หมายเหตุ: ต้องระบุ `UnityEngine.Random` ชัดเจนเพื่อหลีกเลี่ยง `conflict` กับ `System.Random`)
- แสดงผลข้อความบทสนทนาที่สุ่มได้ออกมาทาง Console

Test Cases:

1. **Input:** `as08_dialogues = ["สวัสดีครับ", "คุณเป็นอย่างไรบ้าง", "มีอะไรให้ช่วยไหม"]` **Expected Output:** หนึ่งใน 3 ข้อความที่กำหนด
2. **Input:** `as08_dialogues = ["Hello there!", "How are you?", "What can I do for you?"]` **Expected Output:** หนึ่งใน 3 ข้อความที่กำหนด

AS09_MultiplicationTable

วัตถุประสงค์: สร้างตารางสูตรคูณแม่ `as09_n` จาก 1 ถึง 12

Method Signature:

```
void AS09_MultiplicationTable()
```

ตัวแปร (Inspector Fields):

- `as09_n (int)` - แม่สูตรคูณที่ต้องการสร้าง

Logic ที่ต้อง implement:

- สร้าง `for` loop จาก `i = 1` ถึง `i <= 12`
- แสดงผลในแต่ละรอบในรูปแบบ `Debug.Log($"{as09_n}x{i}={as09_n * i}");`

Test Cases:

1. **Input:** `as09_n = 5` **Expected Output:**

```
5x1=5
5x2=10
5x3=15
5x4=20
5x5=25
5x6=30
5x7=35
5x8=40
5x9=45
5x10=50
5x11=55
5x12=60
```

2. **Input:** `as09_n = 1` **Expected Output:**

```
1x1=1
1x2=2
...
1x12=12
```

AS10_FindSummationFromZeroToNUsingWhileLoop

วัตถุประสงค์: หาผลรวมของจำนวนเต็มตั้งแต่ 1 (หรือ 0) ถึง `as10_n` โดยใช้ `while` loop

Method Signature:

```
void AS10_FindSummationFromZeroToNUsingWhileLoop()
```

ตัวแปร (Inspector Fields):

- `as10_n (int)` - จำนวนเต็มที่กำหนด

Logic ที่ต้อง implement:

- กำหนดตัวแปร `sum = 0`; และ `i = 1`; (หรือ `i = 0`);
- ใช้ `while` loop ทำงานจนกระทั่ง `i <= as10_n`
 - บวกสะสมค่า: `sum += i`;
 - เพิ่มค่าตัวนับ: `i++`;
- แสดงผลรวมออกมาทาง Console ในรูปแบบ: `Debug.Log($"ผลรวมของ n จาก 1 ถึง {as10_n} คือ {sum}")`;

Test Cases:

1. **Input:** `as10_n = 5` **Expected Output:** ผลรวมของ n จาก 1 ถึง 5 คือ 15
 2. **Input:** `as10_n = 10` **Expected Output:** ผลรวมของ n จาก 1 ถึง 10 คือ 55
 3. **Input:** `as10_n = 100` **Expected Output:** ผลรวมของ n จาก 1 ถึง 100 คือ 5050
-

AS11_SpawnEnemies

วัตถุประสงค์: สร้างศัตรูหลายตัวตามจำนวนใน Array และกำหนดตำแหน่งเรียงกันบนแกน X

Method Signature:

```
void AS11_SpawnEnemies()
```

ตัวแปร (Inspector Fields):

- `as11_enemyHPs (int[])` - อาร์เรย์ของค่า HP ศัตรูแต่ละตัว

- `as11_enemyPrefab` (`GameObject`) - Prefab ของศัตรูที่จะสร้าง

Logic ที่ต้อง implement:

- ใช้ `for` loop วนสร้างศัตรูตามจำนวน `as11_enemyHPs.Length`
- ในแต่ละรอบ `i` (เริ่มที่ 0 ถึง `n - 1`) ให้สร้างศัตรูด้วย `Instantiate(as11_enemyPrefab, new Vector2(i + 1, 0), transform.rotation)`
 - รอบที่ 1 (`i = 0`): ตำแหน่ง `x = 1`
 - รอบที่ 2 (`i = 1`): ตำแหน่ง `x = 2`
 - รอบที่ `n` (`i = n - 1`): ตำแหน่ง `x = n`
- แสดงข้อความตำแหน่งของศัตรูแต่ละตัว: `Debug.Log($"new enemy at position x = {i + 1}");`

Test Cases:

1. **Input:** `as11_enemyHPs = [100]`, `as11_enemyPrefab = GameObject("EnemyPrefab")` **Expected Output:**

```
new enemy at position x = 1
```

2. **Input:** `as11_enemyHPs = [50, 75]`, `as11_enemyPrefab = GameObject("EnemyPrefab")` **Expected Output:**

```
new enemy at position x = 1
new enemy at position x = 2
```

AS12_CountTime

วัตถุประสงค์: นับเวลา / จับเวลาด้วย Coroutine และ `while` loop

Method Signature:

```
IEnumerator AS12_CountTime()
```

ตัวแปร (Inspector Fields):

- `as12_countTime` (`float`) - เวลาที่ต้องการนับ (วินาที)

Logic ที่ต้อง implement:

- กำหนดตัวแปรจับเวลา `float timer = 0f;`
- ใช้ `while` loop ทำงานจนราบใดที่ `timer < as12_countTime`
 - เพิ่มค่าเวลาตามเฟรม: `timer += Time.deltaTime;` (หรือวนรอบทีละช่วงเวลา)
 - แสดงผลเวลาทศนิยม 2 ตำแหน่ง: `Debug.Log($"timer : {timer:F2}");`
 - รอเฟรมถัดไปด้วย `yield return null;`
- เมื่อจบloop ให้แสดงผล: `Debug.Log($"End timer : {as12_countTime}");`

Test Cases:

1. **Input:** `as12_countTime = 0.0f` **Expected Output:**

```
End timer : 0
```

2. **Input:** `as12_countTime = 1.0f` **Expected Output:**

```
timer : 0.10
...
timer : 1.00
End timer : 1
```

AS13_SumOfNumbersInRow

วัตถุประสงค์: หาผลรวมของตัวเลขในแถว (Row) ที่ระบุของ 2D Array

Method Signature:

```
void AS13_SumOfNumbersInRow()
```

ตัวแปร (Inspector Fields):

- `as13_matrix` (`Grid2DInt`) - 2D array ที่เก็บตัวเลข กรอกค่าเป็นตารางได้จาก Inspector เรียก `as13_matrix.Get2DArray()` เพื่อแปลงเป็น `int[,]`
- `as13_row` (`int`) - ดัชนีของแถว (Row) ที่ต้องการหาผลรวม

Logic ที่ต้อง implement:

- แปลง `as13_matrix` เป็น `int[,]` ด้วย `as13_matrix.Get2DArray()`
- ใช้ `matrix.GetLength(1)` เพื่อหาจำนวนคอลัมน์
- ใช้ `for` loop วนบวกตัวเลขทุกตัวในแถว `as13_row`: `sum += matrix[as13_row, col];`
- แสดงผลรวมออกมาทาง Console (`Debug.Log(sum);`)

Test Cases: `as13_matrix` ตั้งค่าเริ่มต้นเป็น `{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}`

1. **Input:** `as13_row = 0` **Expected Output:** 6 (1 + 2 + 3)

2. **Input:** `as13_row = 1` **Expected Output:** 15 (4 + 5 + 6)

3. **Input:** `as13_row = 2` **Expected Output:** 24 (7 + 8 + 9)

AS14_SumOfNumbersInColumn

วัตถุประสงค์: หาผลรวมของตัวเลขในคอลัมน์ (Column) ที่ระบุของ 2D Array

Method Signature:

```
void AS14_SumOfNumbersInColumn()
```

ตัวแปร (Inspector Fields):

- `as14_matrix` (`Grid2DInt`) - 2D array ที่เก็บตัวเลข กรอกค่าเป็นตารางได้จาก Inspector เรียก `as14_matrix.Get2DArray()` เพื่อแปลงเป็น `int[,]`
- `as14_column` (`int`) - ดัชนีของคอลัมน์ (Column) ที่ต้องการหาผลรวม

Logic ที่ต้อง implement:

- แปลง `as14_matrix` เป็น `int[,]` ด้วย `as14_matrix.Get2DArray()`
- ใช้ `matrix.GetLength(0)` เพื่อหาจำนวนแถว
- ใช้ `for` loop วนบวกตัวเลขทุกตัวในคอลัมน์ `as14_column`: `sum += matrix[row, as14_column];`
- แสดงผลรวมออกมาทาง Console (`Debug.Log(sum);`)

Test Cases: `as14_matrix` ตั้งค่าเริ่มต้นเป็น `{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}`

1. **Input:** `as14_column = 0` **Expected Output:** 12 (1 + 4 + 7)
2. **Input:** `as14_column = 1` **Expected Output:** 15 (2 + 5 + 8)
3. **Input:** `as14_column = 2` **Expected Output:** 18 (3 + 6 + 9)

AS15_MakeTheTriangle

วัตถุประสงค์: สร้างรูปสามเหลี่ยมดาว (*) ด้วย Nested Loop ตามขนาดความสูง `as15_size`

Method Signature:

```
void AS15_MakeTheTriangle()
```

ตัวแปร (Inspector Fields):

- `as15_size` (`int`) - ความสูง / ขนาดของรูปสามเหลี่ยม

Logic ที่ต้อง implement:

- ลูปภายนอกควบคุมจำนวนแถว: `for (int i = 1; i <= as15_size; i++)`
- ลูปภายในสร้างดาวในแต่ละแถว: `for (int j = 1; j <= i; j++)` รวมดาวจำนวน `i` ดวง
- แสดงผลดาวในแต่ละแถวออกมาทาง Console

Test Cases:

1. **Input:** `as15_size = 3` **Expected Output:**

```
*  
**  
***
```

2. Input: `as15_size = 5` Expected Output:

```
*  
**  
***  
****  
*****
```

AS16_MultiplicationTableOf_2_3_and_4

วัตถุประสงค์: แสดงตารางสูตรคูณแม่ 2, 3 และ 4 (คูณ 1 ถึง 12) ในรูปแบบตาราง 3 คอลัมน์โดยใช้ Nested Loop

Method Signature:

```
void AS16_MultiplicationTableOf_2_3_and_4()
```

Logic ที่ต้อง implement:

- ลูปภายนอกวนรอบตัวคูณ `i` จาก 1 ถึง 12 (แต่ละแถว)
- ลูปภายในวนแม่สูตรคูณ `j` จาก 2 ถึง 4 (คอลัมน์)
- แสดงผลในรูปแบบ `{j} x {i} = {j * i}` โดยคั่นระหว่างคอลัมน์ด้วย `\t` (บรรทัดต้องไม่ลงท้ายด้วย `\t`)
- แสดงผลแต่ละแถวออกมาทาง Console

Test Case:

- **Input:** ไม่มี parameters
- **Expected Output:**

```
2 x 1 = 2    3 x 1 = 3    4 x 1 = 4
2 x 2 = 4    3 x 2 = 6    4 x 2 = 8
2 x 3 = 6    3 x 3 = 9    4 x 3 = 12
2 x 4 = 8    3 x 4 = 12   4 x 4 = 16
2 x 5 = 10   3 x 5 = 15   4 x 5 = 20
2 x 6 = 12   3 x 6 = 18   4 x 6 = 24
2 x 7 = 14   3 x 7 = 21   4 x 7 = 28
2 x 8 = 16   3 x 8 = 24   4 x 8 = 32
2 x 9 = 18   3 x 9 = 27   4 x 9 = 36
2 x 10 = 20  3 x 10 = 30  4 x 10 = 40
2 x 11 = 22  3 x 11 = 33  4 x 11 = 44
2 x 12 = 24  3 x 12 = 36  4 x 12 = 48
```

Extra Assignment Method (ไม่บังคับ)

EX_01_TicTacToeGame_TurnPlay

วัตถุประสงค์: จำลองเกม Tic-Tac-Toe (XO) ขนาด 3x3 สำหรับการเดินในแต่ละตา พร้อมตรวจสอบผลลัพธ์และพิมพ์สถานะของเกม

Method Signature:

```
void EX_01_TicTacToeGame_TurnPlay()
```

ตัวแปร (Inspector Fields):

- `ex01_board` (Grid2DString) - กระดาน Tic Tac Toe ขนาด 3x3 กรอกราค่าเป็นตารางได้จาก Inspector เรียก `ex01_board.Get2DArray()` เพื่อแปลงเป็น `string[,]`
- `ex01_playerTurn` (string) - ตาของผู้เล่น "X" หรือ "O"
- `ex01_row` (int) - แถวที่ต้องการเล่น (index 0 - 2)
- `ex01_column` (int) - คอลัมน์ที่ต้องการเล่น (index 0 - 2)

Logic ที่ต้อง implement:

- แปลง `ex01_board` เป็น `string[,]` ด้วย `ex01_board.Get2DArray()`
- ตรวจสอบความถูกต้องของการเดิน (Invalid move):
 - หาก `ex01_row` หรือ `ex01_column` อยู่นอกช่วง 0 - 2 หรือช่องดังกล่าวไม่ว่าง (`!= ""`) ให้พิมพ์กระดานเดิมและ Log `>> Invalid move`
- หากช่องว่างถูกต้อง:
 - อัปเดตกระดานในตำแหน่ง `[ex01_row, ex01_column] = ex01_playerTurn`
 - พิมพ์กระดานที่อัปเดตแล้วออกมาด้วยรูปแบบตาราง:

```
-----
| x |   | o |
-----
|   |   |   |
```

```

-----
|   |   |   |
|   |   |   |
|   |   |   |
|   |   |   |
|   |   |   |
-----

```

◦ ตรวจสอบสถานะเกม:

- มีผู้ชนะในแถว, คอลัมน์ หรือเส้นทแยงมุม -> Log ">> X wins!" หรือ ">> O wins!"
- ไม่มีผู้ชนะและกระดานเต็มแล้วทุกช่อง -> Log ">> Draw"
- ยังไม่มีผู้ชนะและยังมีช่องว่างเหลือให้เล่นต่อ -> Log ">> Continue"

สถานะผลลัพธ์ที่เป็นไปได้:

- ">> X wins!" - ผู้เล่น X ชนะ
- ">> O wins!" - ผู้เล่น O ชนะ
- ">> Draw" - เสมอ (กระดานเต็ม)
- ">> Continue" - เกมยังไม่จบ สามารถเดินต่อได้
- ">> Invalid move" - การเล่นผิดกฎ (ลงซ้ำช่องเดิมหรือออกนอกกระดาน)

Test Cases ตัวอย่าง:

1. Valid Move (เล่นต่อได้):

- **Input:** ex01_board วางเปล่า, ex01_playerTurn = "X", ex01_row = 0, ex01_column = 1
- **Expected Output:**

```

-----
|   | X |   |
|   |   |   |
|   |   |   |
|   |   |   |
|   |   |   |
-----

>> Continue

```

2. Row Win (ชนะแถวแนวนอน):

- **Input:** ex01_board = { {"X","X",""}, {"O","O",""}, {"","",""} }, ex01_playerTurn = "X", ex01_row = 0, ex01_column = 2
- **Expected Output:**

```

-----
| X | X | X |
|   |   |   |
| O | O |   |
|   |   |   |
|   |   |   |
-----

```

```
>> X wins!
```

3. Invalid Move (ลงช่องที่ไม่ว่าง):

- **Input:** `ex01_board = { {"X","","O"}, {"","",""}, {"","",""} }, ex01_playerTurn = "O", ex01_row = 0, ex01_column = 2`
- **Expected Output:**

```
-----
| X |   | O |
-----
|   |   |   |
-----
|   |   |   |
-----

>> Invalid move
```

4. Draw (เสมอ):

- **Input:** `ex01_board = { {"X","X","O"}, {"O","O","X"}, {"X","O",""} }, ex01_playerTurn = "X", ex01_row = 2, ex01_column = 2`
- **Expected Output:**

```
-----
| X | X | O |
-----
| O | O | X |
-----
| X | O | X |
-----

>> Draw
```

คำแนะนำในการทำ Assignment

การใช้ Debug.Log()

- ใช้ `Debug.Log()` สำหรับแสดงผลลัพท์
- ระวังการเว้นวรรคและการขึ้นบรรทัดใหม่ให้ตรงกับที่คาดหวัง
- ใช้ `$"..."` สำหรับ string interpolation

การทำงานกับ Arrays

- ใช้ `array.Length` เพื่อหาขนาดของ array
- ระวัง Array Index out of bounds
- ใช้ `array.GetLength(dimension)` สำหรับ 2D arrays

การทำงานกับ Loops

- ใช้ for loop เมื่อทราบจำนวนรอบที่แน่นอน
- ใช้ while loop เมื่อต้องการวนจนกว่าเงื่อนไขจะเป็นเท็จ
- ระวัง infinite loop

การสุ่มใน Unity

- ใช้ `UnityEngine.Random.Range(min, max)` สำหรับการสุ่ม
- ค่า max จะไม่รวมในผลลัพธ์ (exclusive)

การทำงานกับ GameObjects

- ใช้ `Instantiate(prefab, position, rotation)` เพื่อสร้าง GameObject
- ใช้ `gameObject.name` เพื่อดูชื่อของ GameObject

การตรวจสอบและ Debug

วิธีการ Test

1. เรียกใช้ method ที่ต้องการทดสอบ
2. ตรวจสอบ output ใน Console
3. เปรียบเทียบกับผลลัพธ์ที่คาดหวัง

ข้อผิดพลาดที่พบบ่อย

- การนับ Array index ผิด (เริ่มที่ 0)
- การใช้ `<` แทน `<=` ใน loop condition
- การลืมนับบรรทัดใหม่หรือเว้นวรรค
- การใช้ Random ผิด namespace

เทคนิคการ Debug

- ใช้ `Debug.Log()` เพื่อแสดงค่าตัวแปรระหว่างการทำงาน
- ตรวจสอบเงื่อนไขใน loop และ if statement
- ใช้ Visual Studio debugger เพื่อ step through code



หมายเหตุ

- Assignment นี้เน้นการฝึกฝนพื้นฐาน Arrays และ Loops
- แต่ละ method ต้องให้ผลลัพธ์ตรงกับ test case อย่างแม่นยำ
- สามารถ implement method ไหนก่อนก็ได้ ไม่จำเป็นต้องเรียงลำดับ
- หากมีข้อสงสัย ให้ปรึกษาอาจารย์หรือผู้ช่วยสอน

ขอให้โชคดีกับการทำ Assignment! 🎮